Быстрый элитарный многокритериальный генетический алгоритм

Non-dominated Sorting Genetic Algorithm — версия II

Deb, Pratap, Agarwal, Meyarivan — IEEE TEC, 2002

Ластовецкий Дмитрий

Университет ИТМО
Институт математики

2026

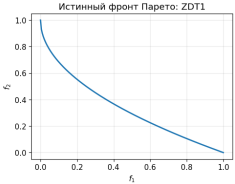
Парето-доминирование и фронт Парето

Доминирование

$$\min_{x \in D} \left(f_1(x), \ldots, f_M(x)\right)$$

$$x \prec y \iff \left(\forall i: \; f_i(x) \le f_i(y)\right) \wedge \left(\exists j: \; f_j(x) < f_j(y)\right)$$

Парето-оптимальное решение — никто из популяции его не доминирует; улучшить по одному критерию нельзя без ухудшения другого.
Фронт Парето — множество всех таких точек в пространстве целей.



ZDT1: выпуклый фронт

ZDT3: разрывный фронт (5 кусков)

Первый NSGA: как он работал

Srinivas и Deb (1994)

Сначала разложить популяцию на недоминируемые фронты, затем присвоить особям из лучших фронтов больший **фитнес** — числовую оценку качества — и размножать их генетическими операторами.

Схема одного поколения:

1. Найти первый недоминируемый фронт $\mathcal{F}_1$.
2. Удалить его, найти $\mathcal{F}_2$, потом $\mathcal{F}_3$ и так далее.
3. Назначить фитнес по рангу фронта: чем лучше ранг, тем выше оценка.
4. Внутри фронта применить ниширование (sharing): штраф за слишком плотную толпу соседей.
5. Сделать отбор, скрещивание и мутацию.

Три слабости

- сортировка $O(MN^3)$;
- нет элитизма: хорошая точка может пропасть;
- нужен ручной параметр ниширования — радиус σ_{share}.

Здесь N — размер популяции, M — число целевых функций.

Три нововведения NSGA-II

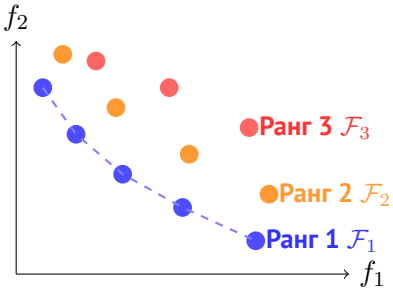
1. **Быстрая недоминируемая сортировка:**
   $O(MN^3) \;\to\; O(MN^2)$ (N — особей в популяции, M — целей)
   При $N{=}100,\ M{=}2$: $2\,000\,000 \to 20\,000$ операций.

2. **Расстояние толпы** (*crowding distance*): автоматическая оценка «незаполненности» окрестности каждой точки — без параметров.

3. **Оператор сравнения $\prec_n$:** единый критерий отбора: ближе к фронту + уникальнее.

Недоминируемые фронты: что это?

Разбиваем популяцию на «слои»: **Ранг 1** — лучшие (никто из популяции их не доминирует). **Ранг 2** — лучшие из оставшихся. И т. д. Меньший ранг = ближе к фронту Парето.



Быстрая недоминируемая сортировка

Для каждой особи p запоминаем:

- n_p — сколько особей *лучше* p (доминируют над ней);
- S_p — список особей, которых p *сама доминирует*.

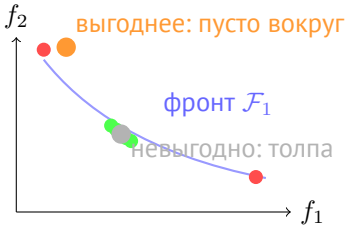
1. Сравнить все пары ☒ заполнить n_p и S_p.
2. $\mathcal{F}_1 = \{p : n_p = 0\}$ (никто не лучше — это ранг 1).
3. Для каждой $p \in \mathcal{F}_1$, для каждой $q \in S_p$: уменьшить n_q на 1. Если $n_q = 0$ ☒ q в $\mathcal{F}_2$.
4. Повторять для $\mathcal{F}_2, \mathcal{F}_3, \ldots$ до конца.

Почему $O(MN^2)$?

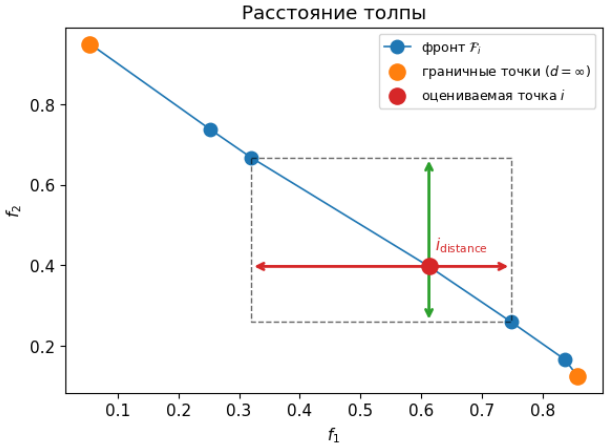
Двойной цикл: $N(N-1)/2$ пар, каждое сравнение $O(M)$. При $N = 100, M = 2$: $\approx 10\,000$ операций. Старый NSGA с $O(MN^3)$: $\approx 2\,000\,000$.

Расстояние толпы — зачем?

На одном и том же ранге нужно предпочитать **более уникальные** решения — те, что находятся в разреженных частях фронта. Иначе все точки стянутся в одну кучу.



Расстояние толпы — иллюстрация



Для каждой точки i: нормированное расстояние до соседей слева и справа. Крайним точкам — ∞ (их выбирать всегда выгодно).

Расстояние толпы — формула

Алгоритм для фронта $\mathcal{I}$ из l точек:

1. Всем: $i_{\text{dist}} = 0$.
2. Для каждого критерия m:
   - Отсортировать по f_m.
   - Крайним: $i_{\text{dist}} = \infty$.
   - Остальным прибавить:

$$i_{\text{dist}} \mathrel{+}= \frac{f_m\big(\mathcal{I}[i{+}1]\big) - f_m\big(\mathcal{I}[i{-}1]\big)}{f_m^{\max} - f_m^{\min}}.$$

Физический смысл

i_{dist} — периметр прямоугольника, построенного на ближайших соседях. Большой периметр = точка в «незаполненном» месте = ценная.

Оператор сравнения с толпой $\prec_n$

Как выбрать лучшего из двух кандидатов?

1. У кого меньший ранг (ближе к фронту) — тот лучше.
2. Если ранги одинаковы — лучше тот, у кого *больше* расстояние толпы (в менее заполненном месте).

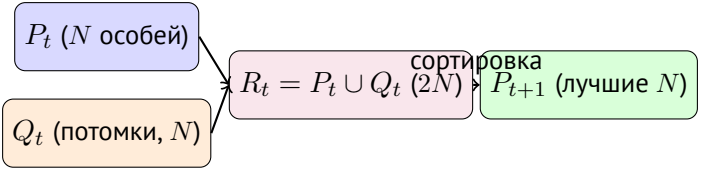
Формально

$$i \prec_n j \iff \begin{cases} i_{\text{rank}} < j_{\text{rank}}, \\ i_{\text{rank}} = j_{\text{rank}} \ \wedge \ i_{\text{dist}} > j_{\text{dist}}. \end{cases}$$

В **бинарном турнире**: случайно берём двух особей, побеждает лучший по $\prec_n$.

Элитизм — как реализован

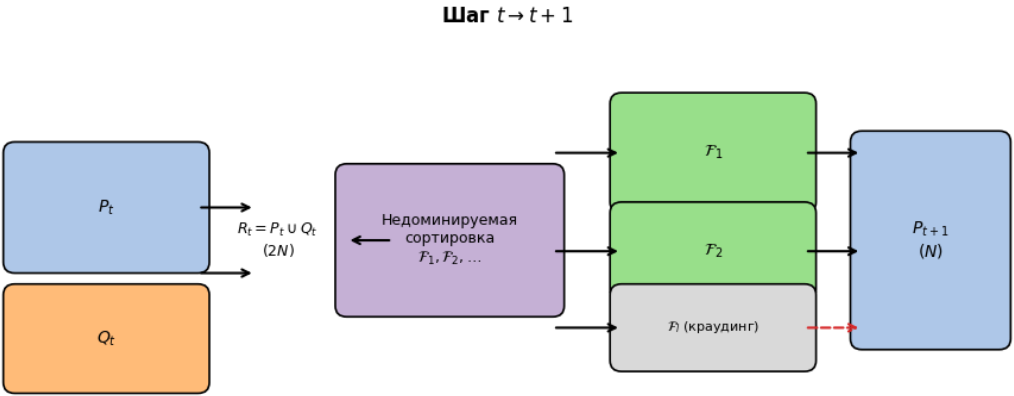
Перед отбором *объединяем* родителей P_t и потомков Q_t. Из объединения $R_t = P_t \cup Q_t$ (размер $2N$) выбираем лучшие N. Если ни один потомок не лучше родителя — родитель остаётся. Хорошие решения **гарантированно не теряются**.



Главный цикл NSGA-II

Шаг	Что происходит
$R_t = P_t \cup Q_t$	Объединить родителей и потомков ☒ $2N$ особей
$\mathcal{F} = \texttt{fast-non-dominated-sort}(R_t)$	Разбить R_t на ранги $\mathcal{F}_1, \mathcal{F}_2, \ldots$
Добавлять $\mathcal{F}_1, \mathcal{F}_2, \ldots$ в P_{t+1}	Брать целые фронты, пока не наполним N мест
Последний фронт: сортировать по $\downarrow i_{\text{dist}}$	Из «пограничного» фронта взять самые уникальные
$Q_{t+1} = \texttt{make-new-pop}(P_{t+1})$	Турнир $\to$ скрещивание $\to$ мутация

Схема одного поколения



Как создаются потомки: SBX и мутация

SBX (Simulated Binary Crossover — симулированное бинарное скрещивание)

Берём двух родителей и создаём двух потомков, находящихся «вокруг» родителей. Параметр η_c: большой — потомки близко к родителям (точный поиск); малый — далеко (широкий поиск). В статье: $\eta_c = 20$.

Полиномиальная мутация

Случайное малое изменение одной переменной потомка с вероятностью $p_m = 1/n$. Позволяет исследовать точки, недостижимые только скрещиванием. В статье: $\eta_m = 20$.

Сложность — что значит $O(MN^2)$

Обозначения

N — размер популяции, то есть сколько решений сравниваем в одном поколении.
M — число целевых функций, например цена/качество/вес дают $M = 3$.

$O(MN^2)$ означает: количество операций $\approx M \times N^2$.
При удвоении N работа растёт в 4 раза. У старого NSGA ($O(MN^3)$): в 8 раз — т. е. в N раз медленнее.

Шаг	Сложность	При $N = 100, M = 2$
Быстрая сортировка	$O(MN^2)$	$\approx 20\,000$ оп.
Расстояние толпы	$O(MN \log N)$	$\approx 1\,300$ оп.
Отбор из $\mathcal{F}_l$	$O(N \log N)$	≈ 700 оп.
Итого	$\boldsymbol{O(MN^2)}$	$\approx 20\,000$ оп.
NSGA (старый)	$O(MN^3)$	$\approx 2\,000\,000$ оп.

Тестовые функции: что такое ZDT1, ZDT2, ZDT3

ZDT — стандартный набор синтетических задач Zitzler–Deb–Thiele. Они нужны не ради прикладного смысла, а чтобы проверить алгоритм на фронтах разной формы.

Задача	n	**Фронт**	**Зачем нужна**
SCH (Schaffer)	1	простой выпуклый	Игрушечная проверка: две цели x^2 и $(x-2)^2$, истинный фронт легко увидеть.
ZDT1	30	выпуклый	Базовый тест: надо свести $x_2, \ldots, x_n$ к нулю и равномерно покрыть гладкую кривую.
ZDT2	30	невыпуклый	Проверяет, не «проваливается» ли метод на вогнутом участке фронта.
ZDT3	30	разрывный	Самый наглядный пример: фронт состоит из нескольких отдельных островков, алгоритм должен найти все.
ZDT4	10	много локальных ловушек	Проверяет устойчивость к ложным фронтам: легко застрять далеко от истинного решения.

Во всех ZDT первые координаты задают форму фронта, а функция $g(x_2, \ldots, x_n)$ штрафует уход от настоящего фронта. Поэтому алгоритм должен одновременно приблизиться к фронту и распределиться по нему равномерно.

Структура ZDT: как устроены функции

Общая форма: $f_1 = x_1, \quad f_2 = g(x_2, \ldots, x_n) \cdot h(f_1, g), \quad x_i \in [0, 1], n = 30.$

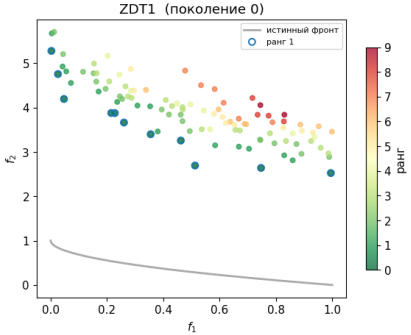
g штрафует за отклонение «лишних» переменных $x_2, \ldots, x_n$ от нуля. На истинном фронте $g = 1$.

	$g(x_2, \ldots, x_n)$	$h(f_1, g)$
ZDT1	$1 + \frac{9}{n-1} \sum_{i=2}^{n} x_i$	$1 - \sqrt{f_1/g}$ выпуклый фронт
ZDT2	то же	$1 - (f_1/g)^2$ невыпуклый фронт
ZDT3	то же	$1 - \sqrt{f_1/g} - \frac{f_1}{g} \sin(10\pi f_1)$ sin создаёт 5 разрывных кусков
SCH	$n = 1, \quad f_1 = x^2, \quad f_2 = (x-2)^2, \quad x \in [-10^3, 10^3]$	

ZDT1: эволюция — поколение за поколением

$$n = 30,\ N = 100,\ T = 250,\ \eta_c = \eta_m = 20$$

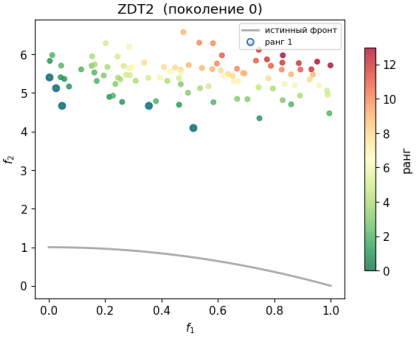
цвет = ранг (синий — ранг 1; красный — хуже)



ZDT2: невыпуклый фронт

$$n = 30,\ N = 100,\ T = 250$$

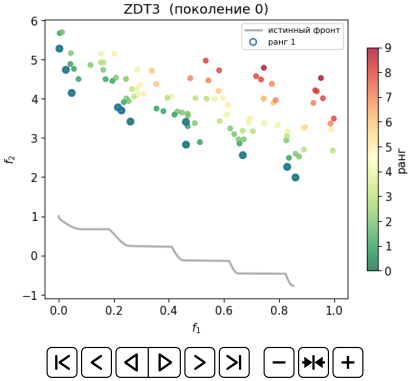
без ручных параметров алгоритм строит невыпуклую кривую



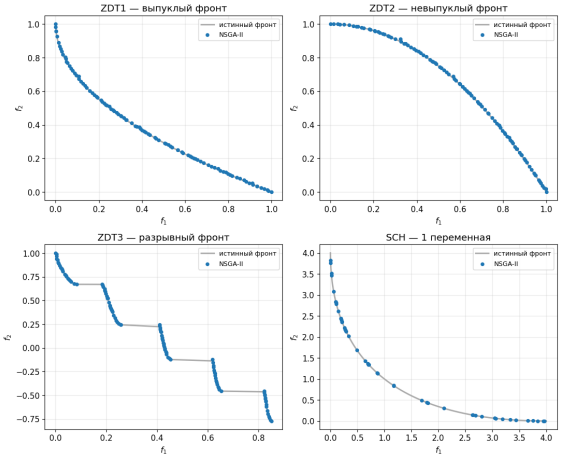
ZDT3: разрывный фронт

$$n = 30,\ N = 100,\ T = 250$$

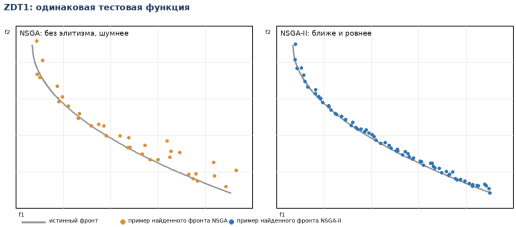
алгоритм самостоятельно «находит» все пять кусков фронта



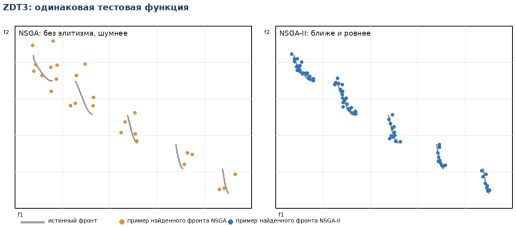
Финальные результаты



NSGA против NSGA-II: одинаковые функции



ZDT1: NSGA-II ближе и ровнее покрывает фронт.



ZDT3: NSGA-II лучше удерживает отдельные куски фронта.

Серые линии — истинный фронт, точки — найденный первый фронт после одинакового бюджета вычислений.

Почему NSGA-II выигрывает визуально

У NSGA

- без элитизма хорошие точки могут исчезнуть;
- ниширование зависит от ручного параметра σ_{share};
- сортировка дороже, поэтому при том же времени меньше полезных поколений.

У NSGA-II

- родители и потомки конкурируют вместе, лучшие сохраняются;
- crowding distance держит точки разнесёнными;
- $O(MN^2)$ вместо $O(MN^3)$.

Как измерять качество

Метрика сходимости Υ — насколько найденный фронт *близко* к истинному.

$$\Upsilon = \frac{1}{|H|} \sum_{h \in H} \min_{x \in \hat{\mathcal{F}}} \|h - x\|_2.$$

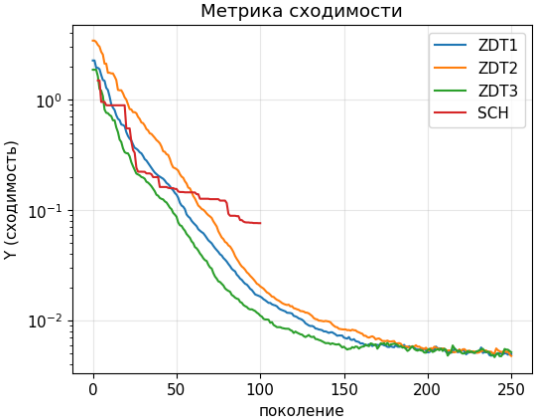
H — эталонные точки на истинном фронте. **Меньше — лучше.** $\Upsilon = 0$ **— точно на фронте.**

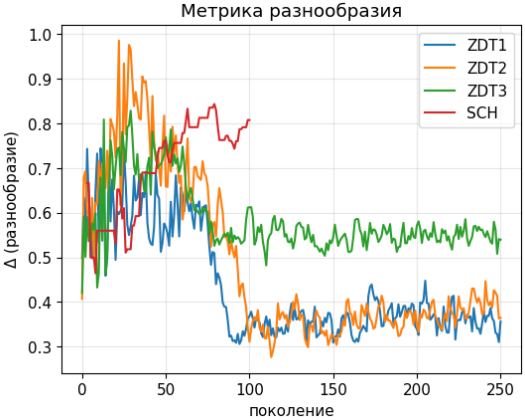
Метрика разнообразия Δ — насколько *равномерно* распределены точки.

$$\Delta = \frac{d_f + d_l + \sum_{i=1}^{N-1} |d_i - \bar{d}|}{d_f + d_l + (N-1)\bar{d}}.$$

d_i — расстояния между соседними точками. $\Delta = 0$ **— идеальная равномерность.**

Кривые сходимости и разнообразия





Сравнение: NSGA и NSGA-II

	NSGA	**NSGA-II**
Сортировка	$O(MN^3)$	$O(MN^2)$
Элитизм	нет: найденная хорошая точка может потеряться	да: выбираем лучших из $P_t \cup Q_t$
Разнообразие	ниширование (fitness sharing) с ручным σ_{share}	расстояние толпы без параметров
Выбор между точками	фитнес после ниширования	ранг + расстояние толпы, оператор $\prec_n$
Практический эффект	медленнее, чувствительнее к настройкам	точнее сходится к фронту, равномернее его покрывает

N — число особей в популяции, M — число целевых функций. NSGA-II устраняет три слабости предшественника: медленная сортировка, отсутствие элитизма и ручной параметр ниширования [1].

Что гарантировано, а что нет

Гарантировано:

- Лучшее найденное решение не ухудшается от поколения к поколению.
- При достаточном числе поколений алгоритм, как правило, находит хорошее приближение к фронту Парето.

Не гарантировано:

- Точное нахождение *всего* фронта Парето — только вероятностно.
- Одинаковый результат при разных запусках (алгоритм случайный).

Теорема «No Free Lunch» [2]: не существует алгоритма, лучшего на всех задачах сразу.

Итоги

1. NSGA-II решает 3 проблемы NSGA: скорость, элитизм, параметр нишинга.
2. $O(MN^2)$ — при $N = 100$ это в 100 раз меньше операций, чем $O(MN^3)$.
3. Расстояние толпы — разнообразие без ручной настройки.
4. Оператор $\prec_n$ совмещает качество (ранг) и уникальность.
5. По сравнению с первым NSGA он быстрее, устойчивее и лучше сохраняет найденный фронт.

Код

`notebooks/nsga2_demo.ipynb`

Ядро алгоритма	`src/nsga2_core.py`
Эксперименты	`notebooks/nsga2_demo.ipynb`
Анимации и графики	`scripts/generate_assets.py`

Литература I

[1] Kalyanmoy Deb и др. “A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II”. В: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), с. 182—197. DOI: 10.1109/4235.996017.

[2] David H. Wolpert и William G. Macready. “No Free Lunch Theorems for Optimization”. В: *IEEE Transactions on Evolutionary Computation* 1.1 (1997), с. 67—82. DOI: 10.1109/4235.585893.